华中科技大学

计算机系统设计

基于 SSD 的过滤向量检索系统
性能测试与设计比较

院　系　计算机科学与技术

班　级　硕 2601 班

姓　名　邓贤昌

学　号　M202674157

目 录

1 实验环境与复现信息 **1**
1.1 实验环境 1
1.2 Debug 验证与复现说明 1
1.3 代码与复现材料 2
2 系统设计理解 **2**
2.1 PipeANN-Filter 2
2.2 GateANN 2
2.3 设计比较 3
3 实验方法 **3**
3.1 数据与工作负载 3
3.2 统一参数与统计口径 3
3.3 主要实验命令 4
4 实验结果与机制分析 **5**
4.1 Recall–Latency 5
4.2 Selectivity–Performance 6
4.3 资源 Trade-off 8
4.4 关键现象与瓶颈总结 8
5 优化方向 **9**
6 实验总结与结论 **13**
附录 A 自主探索与代码改进 **14**
A.1 研究问题与初始观察 14
A.2 假设与代码改动 14
A.3 对照实验设计 15
A.4 结果与消融 16
A.5 发现与适用条件 17
参考文献 **18**

1 实验环境与复现信息

实验在一台 NVMe 服务器上跑：同一份 YFCC 100 万向量数据、同一组查询、相同的 threads/beamwidth/K，只切换 PipeANN-Filter 和 GateANN。正式结果 3 轮共 30 行，文中的数来自这 30 行和我后来补的对照实验。

1.1 实验环境

表 1-1 实验环境与正式实验参数

项目	内容
CPU	Intel Xeon w9-3575X；44 核 / 88 线程；最高 4.8 GHz；单 NUMA 节点
内存	503 GiB；正式实验时可用约 489 GiB
SSD 与文件系统	Samsung SSD 9100 PRO 4TB NVMe；/home 位于 /dev/nvme0n1p5，ext4，1.5 TB 分区；实验前可用约 668 GB
操作系统与内核	Ubuntu 22.04 LTS；Linux 5.15.0-139-generic x86_64
编译器	g++ (Ubuntu 11.2.0-19ubuntu1) 11.2.0；CMake 4.4.4
PipeANN-Filter Commit	2ec84fe20c37dc3cc236b66f7eba31c0cc80bf1e
GateANN Commit	71e87cbf0e25c0400c0109dd25a64a7c4dccc9dd
I/O Backend	Linux AIO（系统调用 O_DIRECT）
公共参数	threads=1，beamwidth=8，K=10，mem_L=0，cache budget=0；GateANN FullAdj neighbors=32
正式实验 experiment-id	formal-yfcc-20261028-01（3 次独立重复）

1.2 Debug 验证与复现说明

数据包和两套 Formal 索引先过了仓库脚本的完整性检查，10 万向量的 Debug 索引也重新构建了一遍。Debug 跑 High、200 条查询、threads=1、beamwidth=8、K=10：PipeANN-Filter L=40 的 Recall@10 是 95.40%，GateANN L=320 是 95.50%，检查器输出 DEBUG CORRECTNESS CHECK PASS。

Formal 索引的校验输出是 Dataset verification passed、Index verification passed，manifest 里记的是 dataset-v1.0、tier formal_1000000。索引采用课程预构建的 100 万向量版本。

正式 30 行沿用课程原版脚本和参数。源码改动是 GateANN 的 AIO 兼容补丁，把 cb.data 改成 cb[j].data，用来通过编译；算法和数据布局保持课程原版。

调试记录主要有三类。第一类是 GateANN 在 AIO 下编译失败，定位到 control block 的 cb.data 应写成 cb[j].data，应用课程补丁后编译通过。第二类是索引和数据版本检查，Formal 的 Dataset verification 与 Index verification 均通过，manifest 的 dataset-v1.0、formal_1000000 与运行参数一致。第三类是时延口径：GateANN 只输出整数 QPS，适配器用 1,000,000/QPS 反推时延，报告统一采用系统 Query Phase 的 mean_latency_us，避免把取整误差当成性能差异。

自主探索放在独立的 worktree 里，改动见附录 A，探索结果与正式 30 行分开记录。

编译用 Release + AIO。每次运行的 command.json、run.log 都留在 artifacts/，汇总结果在 results/。

1.3 代码与复现材料

优化实现、课程补丁、对照脚本和汇总数据统一放在公开仓库：https://github.com/SnpCoder/fvs-course-lab。仓库只保留代码、patch、脚本、汇总 CSV 和报告，不包含数据集、预构建索引与原始运行日志；具体文件改动见附录 A.2。

2 系统设计理解

两套系统都把图索引放在 SSD 上，但过滤发生的位置、过滤后怎么保证图还连得通、内存和 SSD 各放什么，差别很大。下面的描述对应课程固定的源码版本，节点布局和内部计数是我对着日志核过的。

2.1 PipeANN-Filter

PipeANN-Filter 延续 PipeANN [1] 的 SSD-resident 图搜索：原始 uint8 向量、Vamana 邻接和过滤属性一起写进 SSD 节点，查询时按页读；DRAM 只留 PQ 码本和压缩向量、tag 映射、元数据以及近似成员结构。这份 100 万节点索引里，单节点最大 7,080 B，按 8 KiB 对齐读；节点内有 192 B 向量、32 个一跳邻居、224 个额外稠密邻居（R_dense=256）和 5,860 B 属性槽。图节点只保存向量、邻接和属性，标签倒排作为独立的 SSD 文件（约 776 MiB）。

查询先估过滤选择率、近似成员判断的 precision 和候选读取量，再用“SSD 读权重 10、距离计算权重 1”的成本模型在 Pre-filter、In-filter、Post-filter 里选一条。Pre-filter 扫 SSD 标签倒排拿候选，PQ 筛出前 L 个再精确重排；In-filter 在图上遍历，用 DRAM 近似结构提前排除明显不满足的邻居，读节点后用内联属性精确验证；Post-filter 先普通搜索再过滤。Low 档 200 条全走 Pre-filter，Medium/High 全走 In-filter，本次只用到这两条路径。

2.2 GateANN

GateANN [3] 的图更紧凑：每节点 324 B，192 B 向量加 4 B 邻居数和 32 个 4 B 邻居，4 KiB 页放 12 个节点。核心图文件 326 MiB，另有 31 MiB PQ 压缩向量和 126 MiB FullAdjIndex。启动时把 100 万节点的 1,083 万条标签装进 49 MiB 的 SpmatFilterStore，FullAdj 用 MAP_POPULATE mmap 到 DRAM，查询进程 RSS 约 416 MiB。

一次查询从入口点开始维护候选池。对每个候选，GateANN 先读 DRAM 里的完整标签集合做 **Pre-I/O Filtering**：只有满足 selected_label in point_labels 的候选才读 SSD、算精确距离；其余候选从 FullAdj 取 32 个邻居，算 PQ 后继续扩展，这就是 **Graph Tunneling**。只有通过过滤的候选才读 4 KiB 节点页、算精确距离并扩展 SSD 邻居。这样无效节点只消耗 DRAM 邻接和 PQ 计算，随机读留给真正可能命中的节点。代价是 FullAdj 和标签结构常驻内存；强过滤下绝大多数候选都不满足条件，候选池会被无效节点占满，tunneling 的 CPU 时间成了主要成本。

2.3 设计比较

表 2-1 PipeANN-Filter 与 GateANN 的设计对比

系统	过滤查询机制	DRAM 中的主要结构	SSD 中的主要结构	预期优势	潜在代价
PipeANN-Filter	逐查询成本模型在 Pre/In/Post 间路由；Pre 扫描 SSD 标签倒排，In 用近似成员 + 稠密图	PQ 码本/码字、tag、近似过滤结构（实测约 151 MiB RSS）	8 KiB 图节点（含 5,860 B 属性）+ 776 MiB 标签倒排	强过滤 Recall 高；查询 RSS 低	索引 8.43 GiB；单节点 I/O 大；Medium 有额外属性预扫描
GateANN	读前 Pre-I/O Filtering；失败节点用 FullAdj 做 Graph Tunneling	49 MiB 标签 CSR + 126 MiB FullAdj + PQ（实测约 416 MiB RSS）	324 B 紧凑节点；326 MiB 核心图 + 31 MiB PQ	High/Medium 逻辑 I/O 少、时延低；索引小	RSS 高；强过滤 Recall 低；tunneling CPU 成本高

两套系统最核心的差异是过滤位置：PipeANN-Filter 能在读 SSD 前用标签倒排直接出候选，也能在遍历中用近似成员和稠密图边搜边过滤；GateANN 固定读前检查，再用 FullAdj 在无效节点上继续扩展。前者给强过滤留了标签优先的路径，后者把所有过滤放在 I/O 之前，弱/中等过滤省 I/O，强过滤容易把预算耗在无效候选上。资源上，PipeANN 把成本放在 SSD 空间（8.43 GiB）和节点内联属性上，GateANN 放在 DRAM（FullAdj 加标签）上。

3 实验方法

3.1 数据与工作负载

实验用课程数据集 [6] 的 Formal tier：100 万个 192 维 uint8 向量，距离度量 L2，平均每个向量 10.83 个原始标签，全部保留；查询谓词统一为 selected_label in point_labels。每档 5 个真实标签、每个标签 40 条查询，共 200 条；每条查询有过滤后的精确 top-100 Ground Truth，实验取 top-10。

表 3-1 Formal workload 的三档真实标签分桶

Bucket	Selectivity 范围（%）	中位数（%）	查询数	标签数 × 每标签查询数
Low（强过滤）	0.250–0.851	0.545	200	5 × 40
Medium	1.765–4.995	2.958	200	5 × 40
High（弱过滤）	7.074–17.089	12.323	200	5 × 40

Selectivity 表示满足过滤条件的 Base 向量比例。三档使用不同真实标签，桶间差异同时包含标签频率、语义和查询难度的影响。

Debug tier 是 10 万向量，Ground Truth 单独计算，用于构建和正确性检查；Formal 使用 100 万向量的预构建索引和独立 Ground Truth。

3.2 统一参数与统计口径

表 3-2 正式实验统一参数

参数	取值	说明
I/O Backend	Linux AIO	两套系统均使用 O_DIRECT 读取索引
查询线程	1	避免多线程争用 SSD 队列和 CPU
Beam width	8	PipeANN 与 GateANN 相同
Top-K	10	Recall@10

参数	取值	说明
mem_L / cache	0 / 0	不使用内存图索引或热点缓存
GateANN FullAdj neighbors	32	与预构建 FullAdjIndex 一致
独立重复	3	每轮 10 行，共 30 行
PipeANN High L	20、40、80	低成本、代表性、高成本
GateANN High L	160、320、640	课程校准的三档预算
代表性 L	PipeANN 40；GateANN 320	用于 Low/Medium 的固定预算比较

每个 system + bucket + L 跑 3 个独立进程，表和图取三轮中位数，波动用最小值 - 最大值表示。QPS 是整档 200 条查询的进程内统计。PipeANN 原程序直接输出 mean_latency_us；GateANN 只输出整数 QPS，适配器用 1,000,000/QPS 反推平均时延，分辨率受取整影响。两者都只统计 Query Phase，elapsed_seconds 含索引加载和初始化，只作诊断。

mean_ios 只统计 Query Phase 的逻辑读：PipeANN 按 8 KiB 节点算，GateANN 按 4 KiB 页算；比较放在 Recall 相近的点上。PipeANN 日志的 In-filter 列还含属性预扫描，涉及这个差异时写成“日志 In-filter IO”。

peak_rss_kb 是查询子进程整个生命周期的峰值 RSS；索引大小用 index_bytes 单独换算。

正式脚本沿用系统当前缓存状态，索引走 O_DIRECT；每轮里 PipeANN 和 GateANN 顺序跑，避免同时争用 SSD 队列。测试时机器上只跑本实验任务。

3.3 主要实验命令

以下命令均在课程仓库根目录执行。数据与索引下载时，`FVS_DATASET_URL`、`FVS_PIPEANN_INDEX_URL`、`FVS_GATEANN_INDEX_URL` 临时指向 gh-proxy 镜像；其余参数沿用课程配置。源码 fetch 课程固定 commit，再由准备脚本完成 CMake、编译和 doctor。

```
# 数据与固定源码
bash scripts/download_dataset.sh && bash scripts/check_dataset.sh
bash scripts/prepare_pipeann.sh       # 2ec84fe，Release + AIO + tcmalloc
bash scripts/prepare_gateann.sh       # 71e87cb，应用课程 AIO 兼容补丁

# 10 万向量 Debug 索引与正确性验证
python3 adapters/pipeann_adapter.py build --repo "$PWD/systems/PipeANN" \
  --dataset-root "$PWD/datasets/yfcc_course_v1" --tier debug --run-root "$PWD/course_runs" --
threads 8
python3 adapters/gateann_adapter.py build --repo "$PWD/systems/GateANN-public" \
  --dataset-root "$PWD/datasets/yfcc_course_v1" --tier debug --run-root "$PWD/course_runs" --
threads 8
bash scripts/run_debug_validation.sh

# 预构建 Formal 索引与 3 轮正式实验
bash scripts/download_indexes.sh && bash scripts/check_indexes.sh
bash scripts/run_formal_experiments.sh
```

正式脚本每次都会重新校验数据、索引、commit 和编译配置，写 environment.txt，结束时用 collect_results.py 卡 30 行。补充实验用同一套适配器和 thread/beamwidth/K，只改 L，运行 ID 以 supp- 开头，结果单独放在 artifacts/course_runs_results/。

4 实验结果与机制分析

先看 High 档的 Recall–Latency 曲线，再看固定预算下三档 Selectivity 的对比，最后是索引和内存的账。图里的点都是 3 轮中位数，误差线是最小–最大；比时延的时候我都带着 Recall 一起看。

4.1 Recall–Latency

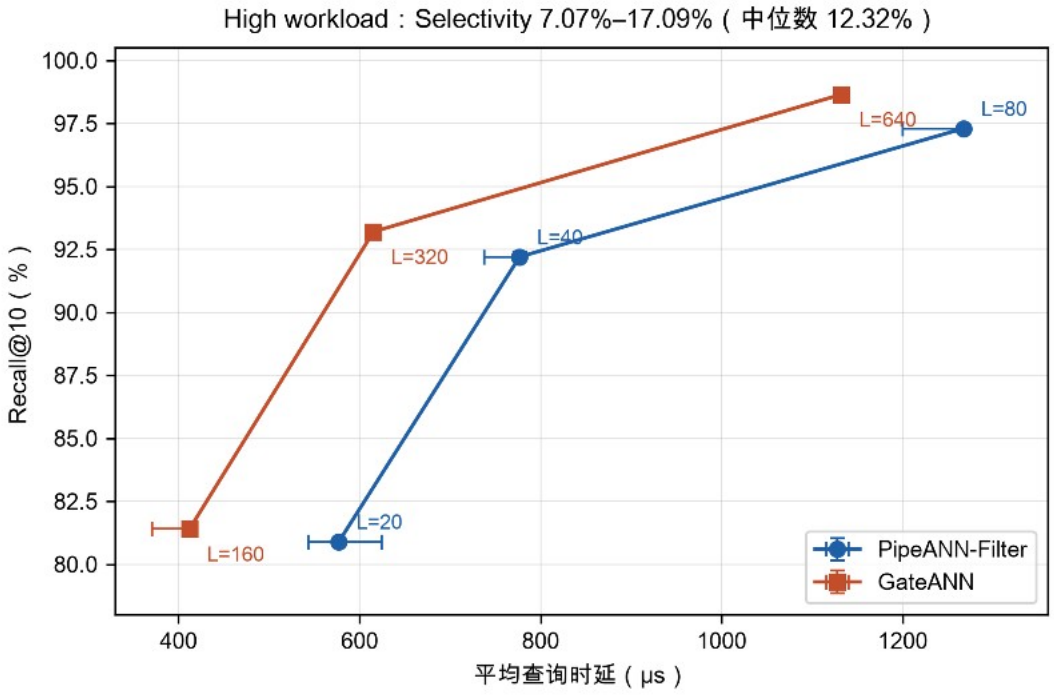


图 4-1 High workload 的 Recall@10–平均查询时延曲线。每套系统连接自己的三档 L；点为三轮中位数，误差线为最小值–最大值。参数：threads=1，beamwidth=8，K=10，AIO。

表 4-1 High workload 下三档搜索预算的结果（3 轮中位数，括号内为最小值–最大值）

系统	L	Recall@10（%）	平均时延（μs）	QPS	逻辑 I/O（次/查询）
PipeANN-Filter	20	80.90（80.80–81.00）	577.2（543.3–624.7）	1719	97.10
PipeANN-Filter	40	92.20（92.20–92.40）	776.8（737.6–777.9）	1280	137.40
PipeANN-Filter	80	97.30（97.30–97.30）	1267.3（1200.2–1270.9）	786	225.00
GateANN	160	81.40（81.40–81.40）	412.2（370.5–413.9）	2426	24.66
GateANN	320	93.20（93.20–93.20）	615.4（608.6–616.5）	1625	46.73
GateANN	640	98.65（98.65–98.65）	1132.5（1126.1–1135.1）	883	88.92

数据来源：results/formal_summary.csv；每个组合 n=3。QPS 与逻辑 I/O 仅列中位数，范围见图 4-1 与 CSV。

加 L 能买到多少 Recall。 PipeANN-Filter 的 L 从 20 加到 80，Recall 从 80.90% 到 97.30%，时延 577.2→1,267.3 μs，逻辑 I/O 97.1→225.0 次；GateANN 从 160 到 640，Recall 81.40%→98.65%，时延 412.2→1,132.5 μs，I/O 24.66→88.92 次。两边都是拿 I/O 换 Recall，但 GateANN 的绝对 I/O 低得多，相对增长反而更快：L 翻 4 倍，I/O 涨 3.6 倍，PipeANN 只涨 2.3 倍。

Recall 相近时比时延和 I/O。 约 81% 处，GateANN（L=160，412.2 µs，24.66 次）比 PipeANN（L=20，577.2 µs，97.1 次）少约 28.6% 时延、74.6% I/O；92–93% 处，GateANN（L=320，615.4 µs，46.73 次）比 PipeANN（L=40，776.8 µs，137.4 次）少 20.8% 时延、66.0% I/O；97–98.65% 处，GateANN（L=640）的 Recall 比 PipeANN（L=80）高 1.35 个百分点，时延低 10.6%，I/O 少 60.5%。三组 Recall 差不多，可以横向比，GateANN 在 High 档一直更低。

这些差异从哪来。 PipeANN 的 High 查询全走 In-filter：L=20/40/80 时分别读约 40.3/64.0/108.6 个候选，近似过滤假阳性 8.4/12.4/19.0 次/查询，日志估计比较量 1,158/2,316/4,633。GateANN 在 L=320 时平均跳过 280.1 个候选、读 46.73 个 4 KiB 页，Tunnel 282 µs、Process 75 µs。它把一部分工作从 SSD 读挪到了 DRAM 标签检查和 FullAdj 扩展上，282 µs 只是 Tunnel 这一段，不含后面的精确距离和堆操作。

4.2 Selectivity–Performance

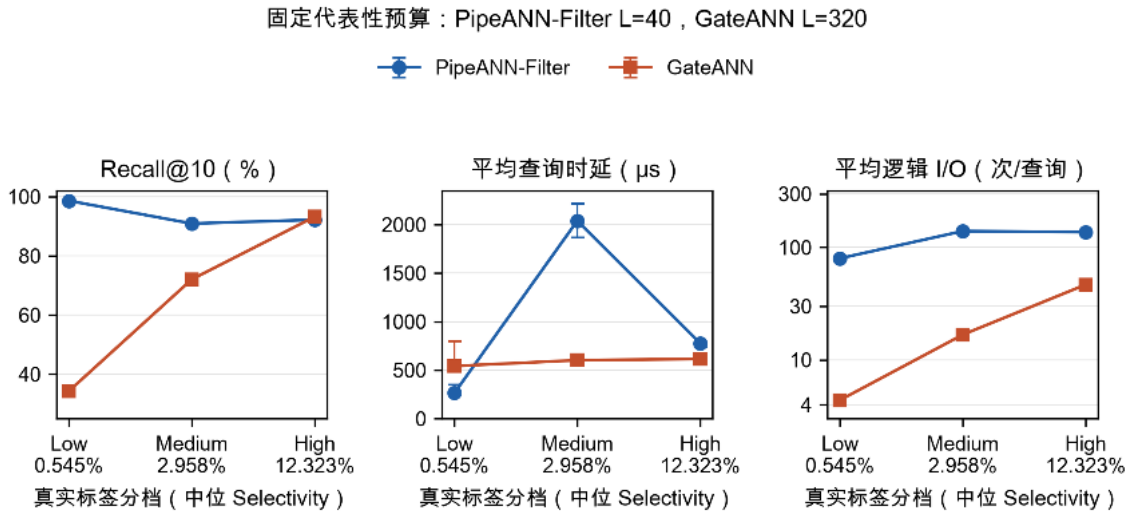


图 4-2 固定代表性预算下的 Selectivity–Performance 组合图。PipeANN-Filter 固定 L=40，GateANN 固定 L=320；三个子图分别为 Recall@10、平均查询时延和平均逻辑 I/O。横轴标注 Low/Medium/High 及其中位 Selectivity；点为三轮中位数，误差线为范围。

图 4-2 三幅子图共用横轴。Low 档 GateANN 的 Recall 明显低，Medium 档 PipeANN 时延最高，High 档两边 Recall 接近，但 GateANN 的时延和 I/O 更低。数在表 4-2。

强过滤下反转。 Low 档 PipeANN 98.60%、263.6 µs、80.0 次；GateANN 34.30%、542.0 µs、4.38 次。GateANN 的 I/O 只有 PipeANN 的 1/18，但 Recall 低 64.3 个百分点——只看 I/O 会得出完全相反的结论。PipeANN 的 200 条 Low 查询全走 Pre-filter：标签倒排给出约 5,490 个候选，假阳性 0，PQ 筛完再精确重排，80 次 8 KiB 读就到 98.6%。

Medium 和 High。 Medium 档 PipeANN 还是 In-filter，Recall 90.90%，时延却涨到 2,039.9 µs，是它三个档里最慢的；GateANN 72.05%、602.4 µs、16.75 次。High 档两边都回来了：PipeANN 92.20%/776.8 µs/137.4 次，GateANN 93.20%/615.4 µs/46.73 次。GateANN 的 Recall 随 Selectivity 从 0.545% 到 12.323% 单调上升（34.30→72.05→93.20），符合“过滤越

弱、有效候选越多”的直觉；PipeANN 因为路由从 Pre 切到 In，反而是 98.60→90.90→92.20 的非单调。

内部计数。 GateANN 的 FilterSkips 是 320.7/309.2/280.1，逻辑 I/O 4.38/16.75/46.73，Tunnel 352/351/282 µs。Low 档几乎每个候选都被跳过，跳过次数贴着 L=320，说明预算都花在无效节点上；Selectivity 上来后，通过的节点多了，I/O 上去、跳过下来，Recall 才恢复。PipeANN 的 Low 全是 Pre（Pre=200、In=0、FP=0），Medium/High 全是 In。Medium 的假阳性 22.8 次/查询，比 High 的 12.4 高；日志 In-filter IO 170.0 次，mean_ios 139.8，差的 30 次是属性预扫描；估计比较量 8,068.8，是 High 同 L 的 3.5 倍。这和 Medium 时延异常对得上。

表 4-2 固定代表性 L 下三档 Selectivity 的结果（3 轮中位数）

系统	Bucket；Selectivity（%）	Recall@10（%）	平均时延（µs）	逻辑 I/O	关键内部指标
PipeANN-Filter	Low；0.250–0.851（0.545）	98.60	263.6	80.00	Pre 200；In 0；FP 0.0
	Medium；1.765–4.995（2.958）	90.90	2039.9	139.80	Pre 0；In 200；FP 22.8
	High；7.074–17.089（12.323）	92.20	776.8	137.40	Pre 0；In 200；FP 12.4
GateANN	Low；0.250–0.851（0.545）	34.30	542.0	4.38	Skips 320.7；Tunnel 352 µs
	Medium；1.765–4.995（2.958）	72.05	602.4	16.75	Skips 309.2；Tunnel 351 µs
	High；7.074–17.089（12.323）	93.20	615.4	46.73	Skips 280.1；Tunnel 282 µs

Selectivity 单元格为最小值 – 最大值（中位数）；关键内部指标为 3 轮中位数。PipeANN 的 Pre/In 是查询数（每档 200），GateANN 的 Skips 为次/查询。

三档用的是不同真实标签，标签频率、查询难度和向量分布都不一样，Selectivity 不是唯一变量。

4.3 资源 Trade-off

表 4-3 索引大小、查询峰值 RSS 与额外结构

系统	索引大小（GiB）	查询峰值 RSS（MiB）	RSS 范围（MiB）	关键额外结构	获得的收益
PipeANN-Filter	8.43	151.1	150.9–151.2	SSD 标签倒排 + DRAM 近似过滤结构；节点内联完整属性	强过滤 Recall 高；查询 RSS 低
GateANN	0.47	416.0	415.9–416.1	49 MiB 标签 CSR + 126 MiB FullAdj	High/Medium I/O 少、时延低；索引紧凑

索引大小由 index_bytes/1024^3 得到；GateANN 的 486 MiB 包含查询时可加载的 FullAdjIndex（126 MiB），核心图文件为 326 MiB。RSS 范围取该系统全部 15 次正式运行的最小–最大值；峰值 RSS 不是索引大小。

两套系统的资源分配几乎相反。PipeANN-Filter 的索引 8.43 GiB，是 GateANN（0.47 GiB）的约 17.8 倍；查询峰值 RSS 约 151.1 MiB，只有 GateANN（416.0 MiB）的 36%。PipeANN 的索引膨胀有明确的实现原因：构建时 AttrIndexWriter::attr_size() 按所有标签中最大的 max_attrs 预留固定属性槽；这份数据里每个向量中位 7 个标签、p99.9 为 194 个、最大 1,462 个，所以单节点属性槽是 $(2 + 1462) \times 4 + 4 = 5{,}860$ B，最大节点 7,080 B，按 8 KiB 对齐后每个节点占一页。100 万节点就是约 8.19 GB（7.63 GiB）的图文件，加 776 MiB 标签倒排和 31 MiB PQ，合计 8.43 GiB。绝大多数向量只有个位数标签，却和最长标签的向量付一样的属性槽，空间放大很明显。

GateANN 把这部分成本转到 DRAM：核心图 326 MiB，但要 49 MiB 标签 CSR 和 126 MiB FullAdj；两者合计 175 MiB，已经能解释 416−151=265 MiB RSS 差值的大部分，其余来自 PQ、页缓冲、元数据和分配器。它的收益是 Medium/High 更少的逻辑 I/O 和更低时延，代价是强过滤下 Recall 崩、常驻内存更高。索引大小和 RSS 的排序正好相反：PipeANN 省内存但吃 SSD，GateANN 省 SSD 但吃内存。

4.4 关键现象与瓶颈总结

Low 档是我花时间最多的现象。GateANN L=320 时 Recall 34.30%，FilterSkips 320.7 次，逻辑 I/O 只有 4.38 次，Tunnel 352 µs——它确实在搜，只是候选在过滤这一步就被刷掉了。我先怀疑是 L 不够，于是在同一索引上补扫 320/640/1,280/2,560：Recall 34.30→83.75%，时延 537.9→3,154.6 µs，FilterSkips 几乎等于 L，逻辑 I/O 只从 4.38 到 25.06。加预算只是让它在无效区域里多走几步，有效候选数量基本不变。

回源码看，GateANN 在 send_best_read_req 里先 check_filter；没通过的节点不读 SSD，而是从 full_adj_ 取 32 个邻居，算完 PQ 后全部插回 retset。L 限制的是 retset 的容量，不是有效候选的容量。Selectivity 0.545% 时，一个无效节点的 32 个邻居里期望只有 0.17 个能过过滤，所以 tunneling 省了 I/O，却把预算继续喂给无效节点。问题在候选预算和可达性，不在 SSD 带宽。

另一个没想到的是 PipeANN 的 Medium：L=40 时时延 2,039.9 µs，是 High 同 L 的 2.6 倍，但 mean_ios 只从 137.4 变成 139.8。日志里 In-filter 读了 170.0 次，估计比较量

8,068.8，假阳性 22.8；High 同 L 是 137.4、2,316.4、12.4。In-filter 在遍历前会 prepare_in_filter 扫冷属性，读多少跟标签分布有关；假阳性高又会把更多候选送进精确验证。Medium 的额外预扫描和 3.5 倍比较量能解释一部分；现有输出只拆到 filter_io_us 和 CPU 等待，2.6 倍里仍有一部分待解释。

两个现象其实是两种资源转移：GateANN 用 DRAM 和 CPU 换 SSD 读，PipeANN 用 SSD 空间和属性预扫描换强过滤 Recall。第 5 节沿 GateANN 这条线做优化，附录 A 用实现和对照来验证。

5 优化方向

我选 GateANN 的强过滤问题继续做。它现象最直接，源码路径也清楚。PipeANN 的属性槽放大留到 4.4，不在这篇里改格式。

Observation

GateANN L=320 的 Low 档：Recall 34.30%、542.0 μs、4.38 次 I/O、FilterSkips 320.7、Tunnel 352 μs。同样数据和参数下，PipeANN L=40 的 Pre-filter 是 98.60%、263.6 μs、80.0 次。为了排除“只是 L 太小”，我只改 L 又跑了 320/640/1,280/2,560 各 3 次：到 2,560 时 Recall 83.75%，时延 3,154.6 μs，Tunnel 2,168 μs，逻辑 I/O 25.06。

补充实验

这组只动 L，数据、索引、thread=1、beamwidth=8、K=10 都和正式实验一样。

表 5-1 Low workload 的补充 L 扫描（3 次独立重复的中位数）

系统	L	Recall@10（%）	平均时延（μs）	逻辑 I/O	FilterSkips/查询	Tunnel（μs）
GateANN	320	34.30	537.9	4.38	320.7	352
	640	51.80	824.4	7.72	637.0	579
	1280	70.05	1557.6	13.95	1270.2	1085
	2560	83.75	3154.6	25.06	2538.5	2168
PipeANN-Filter	20	91.10	215.0	40.00	—	—
	40	98.60	227.6	80.00	—	—
	80	99.90	307.6	160.00	—	—
	160	100.00	492.3	320.00	—	—

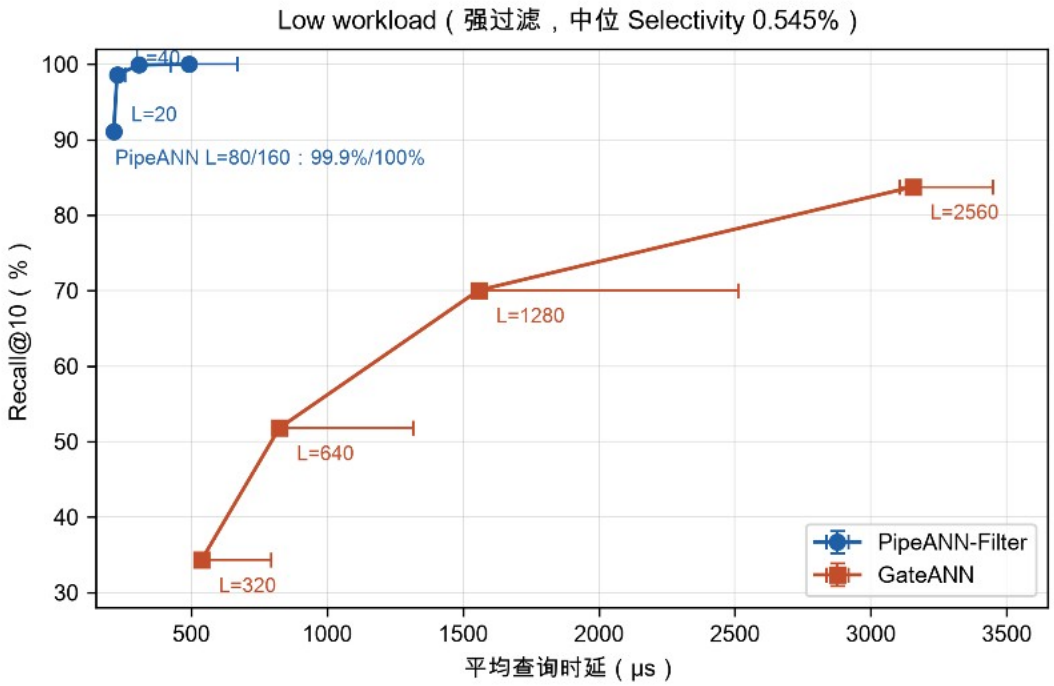


图 5-1 Low workload 下 L 扫描的 Recall@10–平均时延（补充实验）。每套系统使用相同索引和 thread=1、beamwidth=8、K=10；每点 3 次独立重复取中位数，误差线为范围。该图不与图 4-1/4-2 的正式 30 行混合。补充实验运行 ID 为 supp-gate-low-L-sweep-01/02/03 与 supp-pipe-low-L-sweep-01/02/03；参数、索引和数据与正式实验一致，仅 L 不同。GateANN L=320 与正式 Low 点接近但不完全相同，表中使用补充实验自身的中位数。

边际变化很说明问题。L 每翻一倍，Recall 涨 17.5/18.3/13.7 个百分点，时延多 0.29/0.73/1.60 ms，FilterSkips 基本就是 L（320.7→637.0→1,270.2→2,538.5），逻辑 I/O 只从 4.38 到 25.06。钱都花在跳过更多无效候选上。PipeANN 的 Pre 相反：L=20 就 91.10%，L=40 到 98.60%，时延才 215–228 μs，I/O 40–80；再往上主要是把 Recall 推到 100%，I/O 近似线性涨。两边单位 I/O 的有效产出差这么多，是我想做标签优先回退的直接原因。

Mechanism

GateANN 的读前过滤本身没问题：SpmatFilterStore 用 CSR 存每个节点的完整标签，check_filter 在常驻的 49 MiB 上做子集判断，失败节点直接跳过 4 KiB 随机读；FullAdj 保证这些节点还能扩展。问题在于扩展出来的邻居仍然要进同一个候选池，再接受同样的检查。Selectivity 0.545% 时，tunneling 链上绝大多数还是无效节点，L 只是池子容量。

Low 档真正满足条件的向量只有 0.545%×1,000,000≈5,450 个，比 L=2,560 还小一个量级。既然有效集合这么小，直接枚举它比在图上碰运气更合理。PipeANN 的 Pre-filter 就是这么做的：倒排给 ID，PQ 筛，精确重排。强过滤下应该把“标签→向量”的倒排当候选生成器，而不是继续加深 tunneling。

Bottleneck

到这里可以确认：GateANN 的 Low 档瓶颈在候选池，池里缺少足够的有效候选；FilterSkips 贴着 L、I/O 一直很低、Tunnel 时间随时延涨，到 L=2,560 仍未达到 90%。我把瓶颈归到“候选池被无效节点占用、缺少直接的有效候选来源”，SSD 带宽和 FullAdj 都有余量。要验证这个判断，得只改扩展方式——附录 A 的 valid-frontier BFS 就是干这个的：无效节

点进独立 BFS frontier，只有通过过滤的才进 retset。Recall 到了 94.90%，但时延 5.05 ms，每查询检查约 5.1 万个无效节点。

Optimization

方案叫 **Selectivity-Aware Label-First Fallback**：保留 GateANN 的 Pre-I/O Filtering 和 Graph Tunneling 给 Medium/High，只在查询入口加一条强过滤时启用的标签倒排路径。分三步：

- **数据结构**：在构建或启动阶段把 `base.metadata.spmat` 转置成“标签 → 向量 ID”的倒排表，保存每个标签的 offset 和排序后的 ID 数组。按本数据 10,832,801 个 postings、每个 ID 4 B 估算，ID 数组约 41.3 MiB，加 200,386 个 offset（8 B）约 1.5 MiB，合计约 43 MiB。
- **路由**：在查询循环之前用倒排表的标签计数估计 `s = count(label)/N`；当 `s` 低于阈值（初版 1%，验证时扫 0.5%/1%/2%/3%）且查询只含一个标签时走新路径，否则调用现有 `pipe_search`。阈值做成环境变量，方便用 Medium 档检查是否误伤。
- **候选生成与重排**：读取该标签的 posting list，按批用 `nbr_handler->compute_dists` 计算 PQ 距离，维护大小为 L 的最大堆；对堆内候选按 4 KiB 节点页读取并计算精确 L2，返回 top-10。这条路径不经过图遍历，因此不需要 FullAdj，也不需要逐候选 tunneling；多标签查询可以先回退到原路径。倒排表可在启动时从 spmat 转置构建，也可持久化成约 43 MiB 的 sidecar；前者不增加 SSD 空间但增加启动时间，后者相反。

这样只在“强过滤且倒排很小”时换候选来源，Medium/High 继续走原路径，High 档的 I/O 优势保留。

Expected Benefit

收益应该集中在 Low 档。按 5,450 个候选、L=40–80 估，Recall 能从 34.30% 到 90% 以上，目标 95%；时延从 542 µs（L=320）或 3,155 µs（L=2,560）降到 0.3–0.8 ms，逻辑 I/O 40–80 次。PipeANN 的 Pre-filter 在同数据上 L=20/40 是 91.10%/98.60%、215.0/227.6 µs、40/80 次 I/O，说明“枚举约 5,450 个候选 + PQ + 精确重排”这条路走得通；GateANN 的页大小和 PQ 路径不一样，实际能拿到多少要看附录 A 的实现。

Trade-off

代价在内存和查询 CPU。倒排表约 43 MiB，GateANN 的 RSS 会从 416 MiB 涨到约 460 MiB；posting 用 8 B ID 或加计数还会更大。启动时要转置 1,083 万条标签或读 43 MiB sidecar，manifest 也要加版本约束。查询时 5,450 个候选的 PQ 计算比 tunneling 的 4.38 次 I/O 更吃 CPU 和带宽；标签频率估错时，阈值切换可能把 Medium 推到更慢的路径。多标签 AND/OR 要做 posting 交集/并集，初版只开单标签。

Validation Plan

验证用同一台服务器、同一份 Formal 数据和索引，threads=1、beamwidth=8、K=10，三组对照，每组 3 次：

- **A（基线）**：课程原版 GateANN，Low/Medium/High，L=320；直接用正式 30 行中的 GateANN 结果。
- **B（优化）**：GateANN-Hybrid，阈值 1%，Low 档 L=20/40/80 与 L=320；Medium/High 仍走原路径，检查是否有误伤。
- **C（阈值敏感性）**：阈值 0.5%/1%/2%/3%，固定 L=40 或 80，观察 Low/Medium 的 Recall、时延、I/O 和 RSS。

指标是 Recall@10、mean_latency_us、mean_ios、查询峰值 RSS、构建/加载时间和倒排表大小，另外记 posting 长度、PQ 候选数、精确重排数和路由计数。自变量只有“是否启用标签优先”和阈值。我预设的支持条件是：Low 档 Recall≥90%、时延≤1 ms、I/O≤100 次、RSS 增加≤60 MiB，Medium/High 的 Recall 不退、时延变化在 ±10% 内；如果 Low 的 Recall 提升不到 10 个百分点，或者时延超过 542 μs 且 I/O 超过 160 次，或者 Medium 被阈值误判拖慢，就回退阈值或放弃这条路。

这套计划后来在附录 A 实现成了独立分支，实际结果和预期不完全一样，A.4 细说。

6 实验总结与结论

最重要的设计差异在过滤发生的位置。PipeANN-Filter 把过滤做成查询期决策：Pre 从 SSD 标签倒排直接生成候选，In 用 DRAM 近似成员和稠密图边搜边过滤，Post 先搜索再筛选；GateANN 固定在读前检查，失败节点靠 FullAdj 继续扩展。前者用更大的 SSD 索引换强过滤能力，后者用常驻 DRAM 的标签和 FullAdj 换更少的随机读。

最有代表性的结果是两套系统的优势随 Selectivity 反转。High 档 GateANN 在相近 Recall 下时延低两成以上、逻辑 I/O 少六成以上；Low 档 PipeANN 达到 98.60% Recall，GateANN 只有 34.30%，继续加大 L 仍明显落后。

因此 GateANN 更适合 Medium/High 选择率、重视索引体积和 I/O 的场景；PipeANN 适合强过滤、能接受更大索引和单节点读放大、同时要求高 Recall 的场景。

优化方向的核心判断是：强过滤下应把“找有效候选”和“沿图扩展”分开，优先从标签倒排直接生成候选，再走原有精确重排；继续增大 L 的边际收益有限。附录 A 的 seed-only 在 Low 档做到 97.20%，说明这条路径可行；它的限制是单标签和小 posting，代价是额外倒排内存。

这些结论来自单机、每档 200 条查询、3 次重复的实验，三档真实标签不同，物理 I/O 和 per-query 分位数尚未采集，因此更适合解释机制和相对排序，绝对性能还需更多机器和负载验证。对 SSD 驻留的过滤向量检索来说，过滤位置和候选来源比单纯增加 L 更关键，Selectivity 应当和索引布局、缓存策略一起作为系统设计的核心变量。

附录 A 自主探索与代码改进

探索运行都用同一份 Formal 数据、同一个预构建索引，threads=1、beamwidth=8、K=10。改动集中在 GateANN 的查询路径和标签结构：加标签倒排、seed-only 精确重排、valid-frontier BFS 对照；原版路径保留，Medium/High 自动回退。

A.1 研究问题与初始观察

Low 档的结果让我先怀疑候选池，而不是 L。我给自己列了三个问题：

- **候选槽去哪了。** GateANN 在 L=320 时 FilterSkips 约等于 L，逻辑 I/O 只有 4.38 次，Recall 34.30%。这些槽是否被无效节点占满，有效节点难以进入候选池？
- **能不能把无效节点移出主候选池。** 如果不再把 tunneling 遇到的所有邻居插入同一个 L 大小的池，而是用独立的有界 BFS 穿过无效节点、只把通过过滤的节点放进 `retset`，Recall 能否提高？代价是什么？
- **能不能直接生成有效候选。** 如果利用标签倒排直接得到满足单标签谓词的候选，再沿 GateANN 原有的 4 KiB 读页和精确 L2 路径重排，能否在不依赖图遍历的情况下恢复强过滤 Recall？

初始数据支持这个怀疑：每次查询约检查 325 个候选，只有约 1.35% 通过过滤；L 从 320 到 2,560 后 Recall 只到 83.75%，Tunnel 时间从 352 升到 2,168 μs。增加预算主要是在无效区域里多走，有效候选数量基本不变。后两个问题对应两种干预：保留图遍历但改候选池，或者直接更换候选来源。

A.2 假设与代码改动

我从 src/search/pipe_search.cpp 的 send_best_read_req 读起。原版对候选做 check_filter：没通过的不发 SSD 读，而是从 full_adj_ 取 32 个邻居，算 PQ 后把所有邻居（包括无效的）插回 retset。无效节点因此会占 L 个槽，后面的有效节点可能被挤出去。我保留这条路径做基线，另加两条实验路径：

- **valid-frontier BFS（第二个问题的实现）**：失败节点进入独立 FIFO，按 `GATEANN_TUNNEL_HOPS` 做有界 BFS；只有通过 `check_filter` 的节点才插入 `retset`，无效节点只用于继续 BFS。每次失败最多检查 `GATEANN_TUNNEL_MAX_NODES` 个节点，避免无界 CPU。
- **rare-label seed-only（第三个问题的实现，最终方案）**：在 `SpmatFilterStore` 中一次性构建“标签 → 节点 ID”倒排表；单标签且 posting 不大时，取该标签的候选集，按 PQ 距离选前 `L_seed` 个作为 `retset` 的初始候选，只对这些候选做 4 KiB 读页和精确 L2 重排，停止 SSD 邻居扩展。精确过滤判断和距离计算语义保持不变。

表 A-1 自主探索的代码改动

文件/模块	原实现	探索改动
include/utils/spmat_filter_store.h	仅按节点保存 CSR 标签，passes() 做子集判断	新增 build_inverted()、posting()；构建 200,386 个标签、10,832,801 个 postings，约 42.9 MiB
include/ssd_index.h	pipe_search 声明只有过滤参数	增加可选 seed_ids 参数，把标签倒排候选传入查询路径
src/search/pipe_search.cpp	失败节点的一跳邻居全部插入 retset	新增可选 seed_ids、PQ top-L_seed、GATEANN_SEED_ONLY 精确重排；新增 bounded valid-frontier BFS，均由环境变量关闭/开启
tests/search_disk_index_yfcc.cpp	所有 mode=8 查询走原 pipe_search	单标签且 posting≤GATEANN_LABEL_SEED_MAX 时取倒排候选；否则回退原路径；新增 TUNNELSTATS 诊断行
include/utils/percentile_stats.h	只有 n_filter_skip 等计数	新增 n_seed_candidates、n_seed_inserted、n_seed_valid、n_tunnel_valid/invalid，用于消融和开销核算

表 A-1 是全部改动：5 个文件，+294/−28，新增代码分三块——标签倒排、seed 候选与精确重排、valid-frontier BFS；入口只在单标签且 posting 不超过阈值时走 seed，否则原版。新增行为默认关闭。

代码仓库、补丁和对照脚本见 1.3。

A.3 对照实验设计

实验用同一个 100 万向量索引、同一组查询和同一台服务器；每个变体独立跑 3 次，每次 200 条查询。基线是课程原版可执行文件，处理是修改版，每轮里两者顺序跑，避免同时占 SSD 队列。运行结果与正式 30 行分开存放。

对照包括原版 L=320 与 L=640；seed-only 的 L_seed=20/40/60/80（L=320）；seed-only 的 L_seed=40 在 L=640；GATEANN_SEED_ONLY=0 的“种子 + 图扩展”；valid-frontier BFS（H=2，单次最多 512 节点）；以及 Medium/High 的自动回退。Medium/High 的 posting 至少 17,652，超过阈值，因此仍走原路径。

A.4 里 seed-only 的基线是同一轮 explore-base，Low L=320 中位 559.9 µs；正式 30 行的 542.0 µs 是另一批。后文只用 559.9 µs 比。L=1,280/2,560 的基线只用于图 A-1 的参考曲线，来自第 5 节同一索引的 3 轮 L 扫描。

A.4 结果与消融

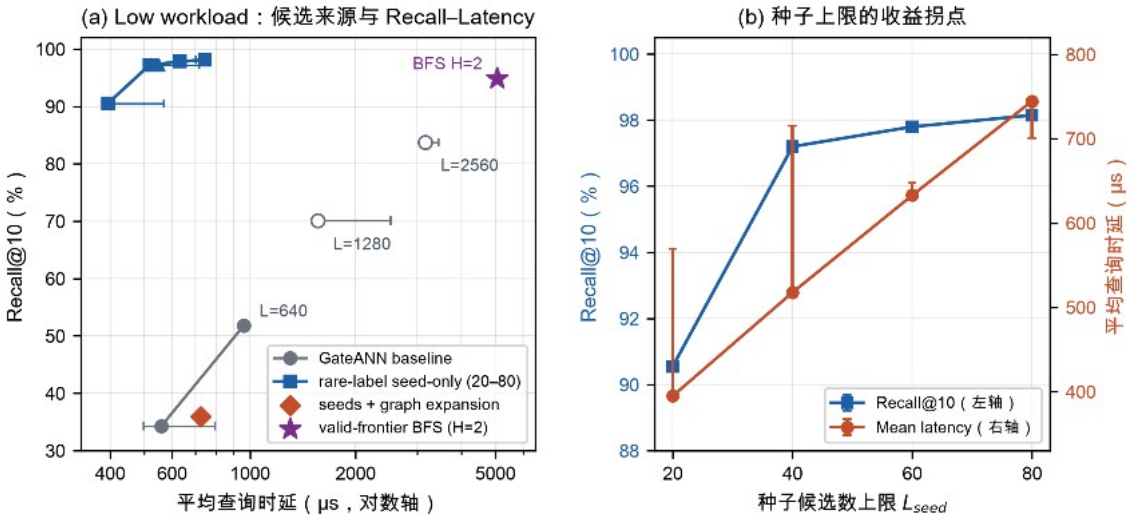


图 A-1 Low workload 下 GateANN 的候选来源、种子预算与消融结果。左图：原版 L 扫描、seed-only、种子+图扩展、valid-frontier BFS 的 Recall–时延；右图：seed-only 的 L_seed 收益拐点。实心点/误差线为 3 轮中位数和范围；基线 L=1280/2560 是独立的 3 轮补充扫描（空心点），不与本次 3 轮对照混合。

seed-only 的结论很清楚：L_seed=40 是这份数据的拐点。此时 Recall 97.20%、中位时延 517.6 μs、逻辑 I/O 39.24；从 40 加到 80 只多 0.95 个百分点，时延和 I/O 接近翻倍。相对原版 L=320，Recall 提高 62.9 个百分点，中位时延略低但两者范围重叠，因此主要收益来自候选来源的变化，几十微秒的中位差不足以说明加速。

只加种子、保留图扩展时，GATEANN_SEED_ONLY=0 的 Recall 是 35.90%，与原版 34.30% 接近。seed 的有效率 39.95/40，说明倒排候选本身有效；但图扩展仍把无效邻居插回容量 40 的池，种子会被距离更近的无效邻居挤掉。这个解释和代码、计数一致，被替换的种子 ID 尚未逐查询记录，因此仍属于推断。

valid-frontier BFS 把无效节点放进独立 FIFO，只让通过过滤的节点进入 retset。Recall 94.90%，但中位时延 5.05 ms、每查询约 5.1 万次无效检查。它说明候选池确实是瓶颈，也说明逐节点 BFS 的 CPU 代价过高；最终方案应直接生成候选。

表 A-2 给出这轮对照的完整数值。

表 A-2 Low workload 的 3 轮对照（中位数；每变体 n=3）

候选路径	L	L_seed	Recall@10（%）	平均时延（μs）	逻辑 I/O	RSS（MiB）	FilterSkips/查询
GateANN 原版	320	—	34.30	559.9	4.38	415.9	320.7
GateANN 原版	640	—	51.80	960.6	7.72	416.1	637.0
seed-only	320	20	90.55	394.8	19.77	416.1	4.8
seed-only	320	40	97.20	517.6	39.24	416.0	14.5
seed-only	640	40	97.25	547.0	39.24	416.0	28.8
seed-only	320	60	97.80	632.9	58.08	416.2	24.1
seed-only	320	80	98.15	744.6	76.19	418.0	33.8
种子 + 图扩展	320	40	35.90	724.6	6.53	416.2	315.8
valid-frontier BFS	320	—	94.90	5050.5	78.95	416.3	51013.6

每个变体 3 次独立重复，表中为 Query Phase 中位数；RSS 为查询子进程生命周期峰值。baseline 559.9 μs 是 explore 批次，正式 30 行的 542.0 μs 是另一个批次。

表 A-3 Medium/High 的回退检查（每变体 n=3，中位数（范围））

Bucket	L	原版 Recall（%）	回退版 Recall（%）	原版时延（µs）	回退版时延（µs）	逻辑 I/O
Medium	320	72.05（72.05–72.05）	72.05（72.05–72.05）	588.93（575.37–691.56）	589.62（567.86–661.38）	16.75
High	160	81.40（81.40–81.40）	81.40（81.35–81.40）	430.85（430.48–434.78）	413.74（412.20–417.01）	24.66
High	320	93.20（93.20–93.20）	93.20（93.20–93.20）	632.11（627.75–634.12）	616.14（607.90–617.67）	46.73
High	640	98.65（98.65–98.65）	98.65（98.65–98.65）	1156.07（1142.86–1160.09）	1150.75（1142.86–1150.75）	88.92

回退版设置了 GATEANN_LABEL_SEEDS=1 、 GATEANN_SEED_ONLY=1 、 GATEANN_SEED_LIMIT=40 ， 但 Medium/High 的 posting 超过 GATEANN_LABEL_SEED_MAX=10000，实际执行原版路径。括号内为三轮最小值 – 最大值；High L=160/320 的回退版时延范围低于原版且不重叠，原因待查，见正文。

回退检查中，Medium/High 的 Recall 中位数与原版一致，说明阈值把查询送回原路径后结果没有变化。High L=160/320 的回退时延低于原版且范围不重叠；这更像两套二进制的布局或分批运行差异，现有证据不足以支持回退版更快的结论。

倒排表的静态大小是 42.9 MiB，但 seed-only 的查询峰值 RSS 从 415.9 到 416.0 MiB，几乎没有变化。比较合理的解释是启动阶段的临时堆被后续分配复用；目前没有做分配器剖析，因此峰值 RSS 不能用来核算这部分内存。

A.5 发现与适用条件

这次探索最确定的结论是：GateANN 的 Low 档瓶颈在候选池，池里缺少足够的有效候选。原版 L=320 时约 320 个槽被无效节点占着，加大 L 只是让 tunneling 走得更远；只把种子塞进图扩展，效果同样有限，因为图扩展会把池子重新填满无效节点。把“找有效候选”和“沿图扩展”分开后，seed-only 才把 Low 档做到 97.20%。

valid-frontier BFS 进一步支持这个判断：它能把 Recall 提到 94.90%，但 5.05 ms 的时延说明逐节点 BFS 不适合作为最终实现；强过滤下更合适的是直接从倒排生成候选。Medium/High 回退后 Recall 与原版一致，说明这条路径可以限制在稀有标签查询里。

适用条件是单标签、posting≤10,000 的稀有标签；多标签 AND/OR 需要 posting 交集/并集。倒排表约 43 MiB，适合内存不紧张的场景；L_seed=40 是这份数据的拐点，不能直接外推到其他数据；阈值 10,000 也取决于 posting 扫描、PQ 比较和 4 KiB 读页的相对成本。

这次探索改变了我对 L 的理解：L 决定候选池容量，但候选来源决定池里有没有有效节点。对 SSD 驻留的过滤向量检索，过滤位置、候选来源和 Selectivity 需要一起设计；单纯增加 L 只能改善遍历覆盖，不能弥补有效候选的缺失。

参考文献

[1] Guo et al. Achieving Low-Latency Graph-Based Vector Search via Aligning Best-First Search Algorithm with SSD. OSDI 2025. https://www.usenix.org/conference/osdi25/presentation/guo

[2] PipeANN-Filter: An Efficient Filtered Vector Search System on SSD. arXiv:2605.17992, 2026. https://arxiv.org/abs/2605.17992

[3] GateANN: I/O-Efficient Filtered Vector Search on SSDs. arXiv:2603.21466, 2026. https://arxiv.org/abs/2603.21466

[4] Subramanya et al. DiskANN: Fast Accurate Billion-point Nearest Neighbor Search on a Single Node. NeurIPS 2019.

[5] Gollapudi et al. Filtered-DiskANN: Graph Algorithms for Approximate Nearest Neighbor Search with Filters. WWW 2023. https://doi.org/10.1145/3543507.3583552

[6] 课程仓库 fvs-course-lab（Gitee），commit e44eab5，2026-09-16. https://gitee.com/wei-fangyu/fvs-course-lab